



南开大学
Nankai University

并行程序设计 Lab3

多线程实验报告

矩阵 SVD 算子 pthread 与 OpenMP 优化

姓名：崔迪

学号：2412439

专业：计算机科学与技术

2026 年 5 月 16 日

目录

1	实验概述	2
2	串行基线与性能瓶颈	2
3	并行优化设计	3
3.1	B 矩阵局部窄带更新	3
3.2	Givens 旋转日志与延迟累积	3
3.3	优化后主循环伪代码	4
3.4	数学等价性分析	6
3.5	pthread 动态任务调度	7
3.6	OpenMP 实现	7
4	正确性验证	8
5	实验环境与方法	8
6	性能结果	8
6.1	大规模样例耗时	8
6.2	结果分析	9
7	同步与数据竞争分析	10
8	Profiling 分析	10
8.1	总体硬件事件统计	10
8.2	热点函数对比	11
8.3	VTune Threading 补充分析	13
8.4	Profiling 结论	14
9	结论	14
A	实验文件说明	15
B	与大模型协作记录	15

1 实验概述

本次 Lab3 选择矩阵 SVD 算子作为多线程实验选题。根据实验指导书，SVD 选题的基础要求包括：完成相应算法的 pthread 与 OpenMP 并行化实现，进行串并行对比、不同问题规模下的性能评估，并讨论任务划分、同步方式、负载均衡、内存访问模式以及 profiling 等问题。

实验框架中，一个一般矩阵 A 首先经 Householder 变换化为上二对角矩阵 B ，随后由 `gkh_svd_from_bidiagonal` 对 B 进行 Golub–Kahan SVD 迭代，最终得到

$$A = U\Sigma V^T.$$

指导书明确指出，本题重点关注 `gkh_svd_from_bidiagonal` 函数。因此，本实验主要优化 GKH 迭代阶段。

2 串行基线与性能瓶颈

原始 GKH 迭代的核心流程如下：每轮先处理对角元接近零的特殊情况，再通过超对角线收敛情况切分 active blocks，最后对每个非平凡 block 做一次 bulge chasing。

算法 1 原始 GKH 迭代主循环

输入：上二对角矩阵 B ，正交矩阵 U, V

```

1: for  $iter = 0$  to  $max\_iter - 1$  do
2:   清理  $B$  中的数值噪声
3:   处理  $d_k \approx 0$  且  $e_k \neq 0$  的特殊情况
4:   根据超对角线  $e_k$  切分 active blocks
5:   if 所有 block 均为  $1 \times 1$  then
6:     迭代收敛，退出
7:   end if
8:   for 每个非平凡 block do
9:     执行一次 Golub–Kahan bulge chasing
10:  end for
11: end for
```

输出：收敛后的对角矩阵 Σ 以及对应 U, V

直接把 block 循环套一层多线程并不是理想方案。其原因有三点：第一，迭代初期通常只有一个大 block，block 级并行度不足；第二，原代码对 B 的行列旋转复用了 dense matrix 更新函数，会对整行或整列扫描，多个 block 并发时存在潜在写冲突；第三，每次 Givens 旋转立即更新 dense 矩阵 U, V ，导致大量跨行的列写入，cache 局部性较差，且并发更新时同步代价较高。

因此，本实验采用的核心优化策略不是简单并行外层 block 循环，而是将 GKH 过程拆成结构稀疏的 B 更新和 dense 的 U, V 批量更新两个阶段。

3 并行优化设计

3.1 B 矩阵局部窄带更新

对于上二对角矩阵上的 bulge chasing, 一次 Givens 旋转只会影响当前 active block 内的小范围元素。原始代码中的 `apply_right_cols(B,...)` 和 `apply_left_rows(B,...)` 会扫描整个矩阵行或列, 造成大量无效访存。

优化后, 将 B 的更新限制在当前 block $[l, r]$ 内:

Listing 1: B 的 block-local 右乘更新

```

1 static void apply_right_cols_B_block(
2     Matrix &B, int c0, int c1, double c, double s, int l, int r)
3 {
4     for (int i = l; i <= r; ++i) {
5         const double a = B.at(i, c0);
6         const double b = B.at(i, c1);
7         B.at(i, c0) = a * c - b * s;
8         B.at(i, c1) = a * s + b * c;
9     }
10 }
```

这样做以后, 不同 active block 对 B 的写入区间互不重叠, 后续 block 级并行时不需要给 B 加全局锁。

3.2 Givens 旋转日志与延迟累积

原代码每产生一个 Givens 旋转, 就立即更新 U 或 V 的两列。对于 1000×1000 的 dense 矩阵, 这类操作是主要内存带宽瓶颈。

本实验将即时更新改为记录旋转日志:

Listing 2: 旋转日志结构

```

1 struct Rot {
2     int a, b;
3     double c, s;
4 };
5
6 struct BlockLog {
7     std::vector<Rot> u_rots;
8     std::vector<Rot> v_rots;
9 };
```

在 `one_block_step_logged` 中, 只更新 B 并记录 U, V 应用的 Givens 旋转。等所有 block 的 B 更新完成后, 再统一将所有旋转按行批量应用到 U, V 。由于矩阵类采用行主序存储, 按行遍历比逐次列更新更符合 cache 访问特征。

3.3 优化后主循环伪代码

综合上述两个核心改造后，优化版 GKH 迭代不再在 block step 中立即更新 dense 的 U, V ，而是将一次迭代拆为三个阶段：block-local 更新 B 、合并旋转日志、并行批量更新 U, V 。伪代码如算法2所示。

算法 2 优化后的 GKH 迭代框架

输入： 上二对角矩阵 B ，正交矩阵 U, V ，最大迭代次数 max_iter ，容差 tol

输出： 对角化后的 B 以及累积后的 U, V

```

1: for  $iter = 0$  to  $max\_iter - 1$  do
2:   处理  $d_k \approx 0$  的特殊情况
3:   根据  $|e_k| \leq tol(|d_k| + |d_{k+1}| + 1)$  切分 active blocks
4:   if 所有 active block 均为  $1 \times 1$  then
5:     标记收敛并退出循环
6:   end if
7:   将非平凡 block 构造成任务，并按长度从大到小排序
8:   清空本轮全局旋转日志  $\mathcal{L}_U, \mathcal{L}_V$ 

9:   并行 block phase:
10:  for all 空闲线程  $t$  do
11:    while 仍有未领取任务 do
12:      原子领取一个 block  $[l, r]$ 
13:      在  $B[l:r, l:r]$  内执行一次 bulge chasing
14:      将产生的左旋转写入本任务的  $\mathcal{L}_{U,[l,r]}$ 
15:      将产生的右旋转写入本任务的  $\mathcal{L}_{V,[l,r]}$ 
16:    end while
17:  end for

18:  按确定的串行 block 顺序合并所有任务日志到  $\mathcal{L}_U, \mathcal{L}_V$ 

19:  并行 dense accumulation phase:
20:  for all 线程  $t$  do
21:    令  $I_t$  为该线程负责的  $U$  的连续行区间
22:    for all  $i \in I_t$  do
23:      将  $\mathcal{L}_U$  中所有旋转依次应用到  $U$  的第  $i$  行
24:    end for
25:  end for
26:  for all 线程  $t$  do
27:    令  $J_t$  为该线程负责的  $V$  的连续行区间
28:    for all  $i \in J_t$  do
29:      将  $\mathcal{L}_V$  中所有旋转依次应用到  $V$  的第  $i$  行
30:    end for
31:  end for
32: end for
33: 将  $B$  的超对角线清零，并对奇异值做非负化与降序重排

```

与原始版本相比，算法2的关键变化是把一次 Givens 旋转的 dense 累积从“立即执行”移动到了每轮迭代的末尾。这样做不会减少必须执行的正交变换数量，但改变了访存模式：原本反复更新 U, V

的两列，变为按行顺序扫描 U, V 并批量执行一串旋转，因而更适合行主序矩阵存储和多线程行划分。

3.4 数学等价性分析

本小节说明优化实现与原串行 GKH 在数学上等价。设某时刻分解关系为

$$A = UBV^T,$$

其中 U, V 为正交矩阵， B 为上二对角或带有局部 bulge 的中间矩阵。

右旋转的等价性。 对 B 的相邻两列右乘 Givens 旋转 R 时，原算法执行

$$B' = BR, \quad V' = VR.$$

由于 R 是正交矩阵，有 $RR^T = I$ ，因此

$$UB'V'^T = U(BR)(VR)^T = UBR R^T V^T = UBV^T = A.$$

所以右旋转只要在 V 中按同样顺序累积，就保持分解不变。优化版只是先把 R 记录到日志中，稍后再执行 $V \leftarrow VR$ ，其代数效果相同。

左旋转的等价性。 对 B 的相邻两行左乘 Givens 旋转 L 时，原算法执行

$$B' = LB, \quad U' = UL^T.$$

同样利用 $L^T L = I$ ，可得

$$U'B'V^T = (UL^T)(LB)V^T = UL^T L B V^T = UBV^T = A.$$

代码中记录到 U 日志的旋转参数为 $(c, -s)$ ，正是因为 U 需要右乘 L^T 而不是 L 。因此，左旋转的延迟累积也保持分解等式不变。

一串旋转的等价性。 设一轮迭代中按串行顺序产生的右旋转为 $R_1, R_2, \dots, R_p$ ，左旋转为 $L_1, L_2, \dots, L_q$ 。原始即时更新最终相当于

$$V_{new} = V_0 R_1 R_2 \cdots R_p,$$

$$U_{new} = U_0 L_1^T L_2^T \cdots L_q^T.$$

优化版将这些 R_i, L_i^T 依次存入日志，并在本轮末尾按相同顺序批量应用，因此得到的 U_{new}, V_{new} 与即时更新完全一致。对于不同 active block，由于 block 的指标区间互不相交，相应旋转作用在不同列集合上；这些嵌入到全矩阵中的正交旋转彼此可交换。实现中仍按原串行从右到左的 block 顺序合并日志，是为了让浮点舍入顺序尽量稳定。

B 的 block-local 更新等价性。 当超对角线 e_k 被判定为收敛并置零后， B 被分解为相互独立的 active blocks。对某个 block $[l, r]$ ，其中的 Givens 旋转可以看成是一个嵌入到全矩阵中的正交矩阵

$$\hat{R} = I_l \oplus R_{[l, r]} \oplus I_{n-r-1},$$

或类似的左乘矩阵 $\hat{L}$ 。全矩阵 dense 更新 $B \leftarrow B\hat{R}$ 与只更新子矩阵 $B[l:r, l:r]$ 的结果相同，因为 block 外对应的行列不是旋转作用区域，block 之间的耦合元素已经为 0。因此，block-local 更新不会改变算法的数学结果，只是避免了对必为 0 或不相关位置的无效访存。

按行并行应用日志的等价性。 批量更新 U 或 V 本质上是右乘一个由 Givens 旋转组成的正交乘积 Q :

$$M_{new} = MQ, \quad M \in \{U, V\}.$$

矩阵乘法按行展开为

$$M_{new}(i, :) = M(i, :)Q.$$

每一行只依赖原矩阵同一行的数据，不依赖其他行。因此，将行集合划分给不同线程并行执行，与串行逐行执行得到相同的数学结果，也不存在多个线程写同一个元素的数据竞争。

综上，优化版并没有改变 Golub–Kahan 迭代施加的正交相似/等价变换序列，只是改变了这些变换在程序中的执行组织方式。因此在忽略浮点舍入微小差异时，优化版与原串行算法等价；实验中的重构误差和正交性误差也验证了这一点。

3.5 pthread 动态任务调度

pthread 版本默认使用 8 个总线程，其中主线程也参与计算，即最多创建 7 个子线程。这与实验指导书中建议的线程组织方式一致。

每轮迭代中，主线程根据 active blocks 构造任务数组，并按 block 长度从大到小排序。各线程通过原子计数器动态领取任务：

Listing 3: pthread 动态任务领取思想

```
1 while (true) {
2     int id = next_task.fetch_add(1);
3     if (id >= tasks.size()) break;
4     one_block_step_logged(B, tasks[id].l, tasks[id].r, tasks[id].log);
5 }
```

任务完成后，主线程按串行版本一致的顺序合并所有 block 的旋转日志，然后所有线程按行区间并行更新 U, V 。这样既避免了频繁创建销毁线程，也避免了对 U, V 加锁。

3.6 OpenMP 实现

OpenMP 版本使用外层 parallel 区域，避免每轮迭代反复 fork-join。初版 OpenMP 曾使用 `omp for schedule(dynamic,1)` 调度 block，并用两个 `omp for schedule(static)` 更新 U, V 。在课程服务器上，该写法的 work-sharing 和隐式 barrier 开销较大。

因此，最终 OpenMP 版本也采用显式 atomic 任务计数器，并手动按线程号划分 U, V 的行区间。同时调用 `omp_set_dynamic(0)` 固定线程数，避免运行时系统动态缩减线程。

Listing 4: OpenMP 版本中的手动任务分发和行划分

```
1 omp_set_dynamic(0);
2 omp_set_num_threads(thread_count);
3
4 #pragma omp parallel num_threads(thread_count)
5 {
6     int tid = omp_get_thread_num();
7     int team_size = omp_get_num_threads();
8 }
```

```

9 // block phase: atomic fetch_add 领取任务
10 // U/V phase: 每个线程处理连续行区间
11 int u_begin = U.rows() * tid / team_size;
12 int u_end = U.rows() * (tid + 1) / team_size;
13 apply_rots_to_rows(U, all_u_rots, u_begin, u_end);
14 }

```

4 正确性验证

测试程序会对 5 组样例进行验证：固定 5×5 矩阵、随机 8×8 矩阵、近秩亏损 10×8 矩阵、随机 10×8 矩阵以及随机 1000×1000 矩阵。每组样例均检查收敛情况、重构误差、 U, V 正交性、对角结构、奇异值降序和非负性。

三种版本在三次服务器测试中均通过全部 5 组样例。以 OpenMP 版本一次 1000×1000 测试为例：相对重构误差为 3.19445×10^{-13} ， $\|U^T U - I\|_F = 2.38537 \times 10^{-13}$ ， $\|V^T V - I\|_F = 2.35243 \times 10^{-13}$ ，对角结构误差为 0，测试结果为 PASS。

表 1: 正确性测试汇总

版本	测试次数	每次样例数	通过情况
serial-origin	3	5	15 / 15 PASS
pthread-optimized	3	5	15 / 15 PASS
OpenMP-optimized	3	5	15 / 15 PASS

5 实验环境与方法

实验使用课程服务器提供的自动测试脚本运行。脚本编译命令为

Listing 5: 测试脚本中的编译命令

```

1 g++ main.cpp gkh.cpp bidiagonalization.cpp -o main "$OPT_FLAG" \
2 -fopenmp -lpthread -std=c++17

```

本次记录的服务器结果均使用随机种子 2412439，编译优化等级为脚本默认或指定的 `-O2`。pthread 与 OpenMP 版本默认总线程数均为 8。

待补充：课程服务器的 CPU 型号、内存规格、GCC 版本、作业调度系统实际分配的节点和核心数。后续可以在提交脚本中加入 `lscpu`、`g++ --version` 等命令，将硬件与编译器信息记录到报告附录。

6 性能结果

6.1 大规模样例耗时

表2 给出随机 1000×1000 样例在三次测试中的耗时。由于本实验只优化 GKH 迭代部分，上二对角化阶段的耗时主要用于说明总耗时构成，不作为优化重点。

表 2: 1000 × 1000 样例三次测试耗时 (单位: ms)

版本	测试	上二对角化	GKH 迭代
serial-origin	1	9471.20	33250.70
serial-origin	2	8695.81	32482.10
serial-origin	3	9005.72	32660.60
pthread-optimized	1	9387.19	7713.29
pthread-optimized	2	7729.00	5174.41
pthread-optimized	3	8808.17	5379.28
OpenMP-optimized	1	7310.88	5746.41
OpenMP-optimized	2	7900.69	5426.43
OpenMP-optimized	3	9192.05	5252.86

三次测试取平均后, GKH 阶段的性能如表3 所示。

表 3: 1000 × 1000 样例平均耗时与加速比

版本	上二对角化均值	GKH 均值	GKH 加速比	总耗时加速比
serial-origin	9057.58	32797.80	1.00	1.00
pthread-optimized	8641.45	6088.99	5.39	2.84
OpenMP-optimized	8134.54	5475.23	5.99	3.08

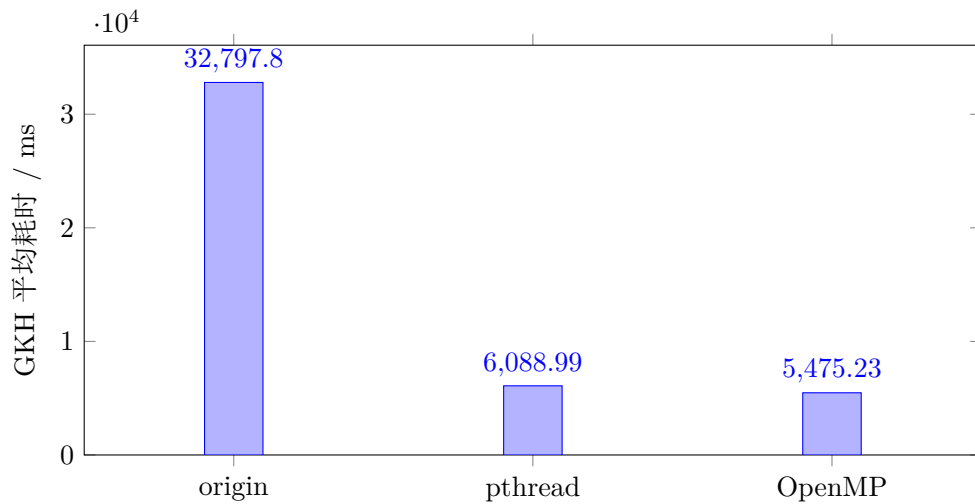


图 1: 1000 × 1000 样例 GKH 阶段平均耗时对比

6.2 结果分析

从表3 可以看出, pthread 版本在 GKH 阶段达到约 5.39× 加速, OpenMP 版本达到约 5.99× 加速。OpenMP 版本最终略快于 pthread, 主要原因是经过修改后 OpenMP 版本避免了每轮多次 omp for work-sharing 调度, 改用接近 pthread 的 atomic 任务领取和手动行划分。

总耗时加速比低于 GKH 加速比，这是因为 `to_bidiagonal` 所在的上二对角化阶段没有被并行优化。在优化后版本中，GKH 阶段已经从原来的约 32.8s 降到约 5.5–6.1s，而上二对角化仍需约 8–9s，成为总耗时中的重要部分。因此，如果继续优化，下一步应考虑 Householder 上二对角化中的矩阵-向量乘与 rank-1 更新。

小规模样例中并行版本不一定更快。原因是 5×5 、 8×8 、 10×8 的实际计算量很小，线程同步、barrier、任务构造和日志合并的固定开销已经超过了计算收益。这也说明本实验优化主要面向大规模 dense SVD 场景。

7 同步与数据竞争分析

本实验中需要重点避免 B, U, V 三类矩阵上的并发写冲突。

对于 B ，`split_active_blocks` 根据已经收敛为 0 的超对角线将问题切分为若干闭区间 $[l, r]$ 。优化后的 GKH step 只在 block 内更新 $B[l:r, l:r]$ ，不同 block 之间写入范围不重叠，因此 block phase 不需要对 B 加锁。

对于 U, V ，每个 block 产生的旋转只涉及该 block 内的相邻列。数学上，不同 block 对应的列集合互不相交。实现上进一步采用旋转日志和按行批量更新：每个线程只负责 U, V 的一段连续行，因此任意两个线程不会写同一个矩阵元素。这样避免了对 dense U, V 的列锁或全局锁。

同步点主要包括：block phase 结束后等待所有任务完成、合并 rotation logs、 U 更新结束后再更新 V 。pthread 版本用自定义 barrier 实现，OpenMP 版本用外层 parallel 区域中的 barrier 与 single 实现。

8 Profiling 分析

为进一步分析优化效果，本实验对 serial-origin、pthread-optimized 和 OpenMP-optimized 三个版本使用 Linux perf 工具进行了 profiling。总体计数使用 `perf stat -r 3 -d -d -d` 重复运行三次，热点函数使用 `perf record` 与 `perf report --stdio` 生成文本报告。当前目录中已保存的文件包括：`perf_stat_origin.txt`、`perf_report_origin.txt`、`perf_stat_pthread.txt`、`perf_report_pthread.txt`、`perf_stat_openmp.txt` 以及 `perf_report_openmp.txt`。

需要特别说明的是，受课程平台使用方式限制，profiling 只能在登录节点完成，无法像自动测试一样申请到独占或较稳定的计算资源。因此，`perf stat` 中的 elapsed time 不用于计算或比较最终加速比；本文主要使用 profiling 的热点函数分布、指令与周期计数、cache/TLB miss rate 以及同步相关函数占比来解释性能瓶颈和优化效果。最终性能和加速比仍以前文自动测试脚本在课程服务器上的三次运行结果为准。

8.1 总体硬件事件统计

表4 总结了三种版本的主要硬件事件。需要说明的是，perf 统计的是完整程序，包括上二对角化、GKH 迭代以及正确性检查中的矩阵乘法，因此它和前文单独统计的 GKH 时间不是同一个口径；表中的 elapsed time 只作为 profiling 运行记录保留，不作为性能对比依据。

表 4: perf stat 总体硬件事件统计（三次运行平均）

版本	elapsed(s)	cycles(10^9)	instr.(10^9)	IPC	L1D miss	dTLB miss
serial-origin	48.915	121.08	118.95	0.98	4.64%	2.19%
pthread-optimized	46.335	93.90	98.94	1.05	1.72%	0.79%
OpenMP-optimized	101.600	101.71	186.15	1.83	1.26%	0.54%

从表4 可以看到, pthread 优化版相比串行版显著减少了周期数和指令数: 周期数从 121.08×10^9 降至 93.90×10^9 , 指令数从 118.95×10^9 降至 98.94×10^9 。这说明优化并不只是依靠多线程并行执行, 还通过 B 的 block-local 更新、删除全矩阵 cleanup 和延迟累积 U/V 减少了实际工作量。

内存层次方面, pthread 版本的 L1 data cache miss rate 从 4.64% 降至 1.72%, dTLB miss rate 从 2.19% 降至 0.79%。这与优化设计相吻合: 原始版本反复对 dense 矩阵做整列更新, 访存跨步较大; 优化后将 U, V 的更新改为按行批量应用旋转, 访问模式更加连续, 因此 L1D 和 dTLB 行为明显改善。

OpenMP 版本的 L1D miss rate 和 dTLB miss rate 也较低, 分别为 1.26% 和 0.54%。但是当前 perf_stat_openmp.txt 显示 elapsed time 为 101.60s, 明显慢于自动测试中最终 OpenMP 版本的运行时间。该现象主要来自登录节点共享环境、线程调度状态和 perf 采样开销, 与自动测试的运行环境并不一致; 因此本文保留 OpenMP 的硬件事件和热点分布作为辅助分析, 但最终性能结论仍以前文三次自动测试的时间为准。

8.2 热点函数对比

serial-origin 的 perf report 显示, 程序最主要热点在 GKH 阶段。gkh_svd_from_bidiagonal 占 59.03%, 其中 one_block_step 占 40.04%。进一步展开可见, apply_right_cols 自身占 39.57%, cleanup_bidiagonal 占 12.35%, apply_left_rows 占 6.48%。这些数据说明原始代码的主要瓶颈正是对 dense B, U, V 的整行/整列旋转更新, 以及每轮全矩阵 cleanup。

表 5: serial-origin 主要热点函数

函数	Children	Self
gkh_svd_from_bidiagonal	59.03%	0.35%
one_block_step	40.04%	0.00%
apply_right_cols	39.57%	39.57%
to_bidiagonal	24.87%	24.87%
Matrix::operator*	16.05%	16.05%
cleanup_bidiagonal	12.35%	12.35%
apply_left_rows	6.48%	6.48%

pthread-optimized 的热点分布发生了明显变化。apply_rots_to_rows 成为最大热点, 占 45.85%, 说明 Givens 日志延迟累积后, dense U, V 的批量行更新成为主要计算工作。run_block_phase 占 11.82%, 其中 one_block_step_logged 占 11.79%, 远低于串行版本中 one_block_step 的 40.04%。这说明 block-local 更新有效压缩了 bulge chasing 的访存范围。

表 6: pthread-optimized 主要热点函数

函数	Children	Self
apply_rots_to_rows	45.85%	45.85%
to_bidiagonal	24.61%	24.60%
Matrix::operator*	16.37%	16.37%
run_block_phase	11.82%	5.01%
one_block_step_logged	11.79%	0.00%
gkh_svd_from_bidiagonal	9.32%	0.17%
apply_left_rows_B_block	6.63%	6.63%

值得注意的是, pthread 版本中 `barrier_wait` 只占 0.22%, `pthread_cond_wait` 只占 0.14%, `merge_task_logs` 约 0.12%。因此, 该实现的同步与日志合并开销较小, 性能主要受 U, V 的批量行更新和未优化的 `to_bidiagonal` 限制。

OpenMP-optimized 的 `perf report` 进一步表明, OpenMP 版本的计算热点和 pthread 版本基本一致: `apply_rots_to_rows` 占 33.37%, 仍是最大单体热点; `one_block_step_logged` 占 10.39%, 其中 block-local 的 `apply_left_rows_B_block` 占 6.76%, `apply_right_cols_B_block` 占 2.26%。此外, `to_bidiagonal` 占 21.60%, `Matrix::operator*` 占 14.76%, 说明未优化的上二对角化和正确性验证中的矩阵乘法仍然占据总程序中的较大比例。

表 7: OpenMP-optimized 主要热点函数

函数	Children	Self
gkh_svd_from_bidiagonal	63.28%	0.00%
apply_rots_to_rows	33.37%	33.37%
to_bidiagonal	21.60%	21.52%
Matrix::operator*	14.76%	14.76%
libgomp runtime addr. 0x1c4ec	12.86%	12.86%
one_block_step_logged	10.39%	3.64%
GOMP_parallel	7.92%	0.00%
apply_left_rows_B_block	6.76%	6.76%
libgomp runtime addr. 0x1c4e0	6.16%	6.16%
apply_right_cols_B_block	2.26%	0.00%

表7 中有两个 libgomp 内部地址未能被 `perf report` 完整解析为函数名, 合计约 19.02%。结合调用栈中出现的 `GOMP_parallel`, 可以判断这部分来自 OpenMP 运行时内部开销或等待开销, 但不能在没有调试符号的情况下进一步精确归因到具体 barrier 或调度函数。这个结果也解释了为什么 OpenMP 的算法热点与 pthread 类似, 但 profiling 的 elapsed time 更不稳定: OpenMP 版本依赖运行时调度和隐式同步, 运行环境变化会更明显地影响整体墙钟时间。

8.3 VTune Threading 补充分析

实验指导书中建议使用 Intel VTune 观察线程同步开销和空闲等待。由于课程服务器未提供交互式 VTune 图形界面，本实验在本地 Windows 机器上使用 VTune 命令行工具对 origin、pthread 和 OpenMP 三个版本补充进行了 **threading** 分析。采集命令如下：

Listing 6: VTune threading 分析命令

```

1 vtune -collect threading \
2   -knob sampling-and-waits=sw \
3   -knob enable-stack-collection=true \
4   -result-dir vtune_results/origin_threading \
5   -- svd_origin.exe 2412439
6
7 vtune -collect threading \
8   -knob sampling-and-waits=sw \
9   -knob enable-stack-collection=true \
10  -result-dir vtune_results/pthread_threading \
11  -- svd_pthread.exe 2412439
12
13 vtune -collect threading \
14   -knob sampling-and-waits=sw \
15   -knob enable-stack-collection=true \
16   -result-dir vtune_results/openmp_threading \
17   -- svd_openmp.exe 2412439

```

本地机器为 20 个逻辑核心，origin 串行版本基本只使用 1 个逻辑核心，pthread 和 OpenMP 版本默认使用 8 个工作线程，且完整程序中还包含串行上二对角化和正确性验证。因此，VTune 的 elapsed time 和 Effective CPU Utilization 不用于评估最终加速比，只用于观察同步与等待行为。最终加速比仍以前文课程服务器自动测试结果为准。

表 8: VTune threading 同步与等待分析摘要

版本	elapsed(s)	Effective CPU Util.	Wait Poor(s)	Spin/Overhead(s)	主要同步或等待函数
origin	26.020	4.9% (0.975/20)	0	0	无明显同步等待; 主要为单线程有效计算
pthread	5.277	5.0% (0.993/20)	11.786	4.388	WaitForMultipleObjects 4.150s; pthread mutex 0.029s; pthread cond wait 0.014s
OpenMP	5.562	4.3% (0.862/20)	34.322	4.935	WaitForMultipleObjects 3.697s; GOMP barrier 0.866s; GOMP work-share end 0.187s

表8 可以补充说明三点。第一，origin 版本的 **Spin and Overhead Time** 为 0，VTune 也没有列出显著的 waiting object。这说明原始串行程序慢并不是因为同步或线程等待，而是因为 GKH 计算本身在单线程上执行，和前文 perf 中 `apply_right_cols`、`cleanup_bidiagonal` 等计算热点占比高的结论一致。

第二, pthread 版本中 `pthread_mutex_lock` 和 `pthread_cond_wait` 的时间很小, 分别只有 0.029s 和 0.014s, 和 Linux perf 中 `barrier_wait`、`pthread_cond_wait` 占比很低的结果一致。这说明 pthread 版本的主要瓶颈不在显式互斥锁竞争, 而在算法阶段之间的等待和 dense U, V 批量更新。

第三, OpenMP 版本中 VTune 明确识别出了 `gomp_team_barrier_wait` 和 `gomp_work_share_end`, 二者分别占 0.866s 和 0.187s。这与 perf 中看到的 libgomp 内部开销相互印证: OpenMP 实现虽然算法结构与 pthread 基本一致, 但仍会承担 OpenMP runtime 的隐式 barrier 和 work-sharing 收尾开销。因此, 最终 OpenMP 版本使用外层 `parallel` 区域、显式 atomic 任务领取和手动行划分, 正是为了减少反复进入 `omp for` 带来的额外调度成本。

8.4 Profiling 结论

综合硬件事件和热点函数结果, 可以得到以下结论:

1. 原始版本的主要瓶颈是 `apply_right_cols`、`apply_left_rows` 和 `cleanup_bidiagonal`。这些函数都体现出对矩阵大范围行列扫描的代价, 验证了优化前关于 dense 更新和全矩阵 cleanup 的瓶颈判断。
2. pthread 优化后, `one_block_step` 的热点占比大幅下降, 说明 B 的 block-local 更新减少了无效访存; 最大热点转移到 `apply_rots_to_rows`, 说明当前主要工作变为 U, V 的批量行更新。
3. OpenMP 版本的热点结构与 pthread 版本一致, 最大热点同样是 `apply_rots_to_rows`, 说明两种实现的核心算法改造相同; 额外出现的 libgomp 与 `GOMP_parallel` 开销则体现了 OpenMP 运行时调度和隐式同步的成本。
4. cache 和 TLB 指标明显改善。pthread 版本 L1D miss rate 从 4.64% 降到 1.72%, dTLB miss rate 从 2.19% 降到 0.79%, 说明按行批量应用旋转比原来的列式即时更新更加符合行主序存储。
5. 同步开销不是 pthread 版本的主要问题。`barrier_wait` 与 `pthread_cond_wait` 占比均低于 0.3%, 表明动态任务领取和阶段式 barrier 的额外开销较小。
6. 优化后的下一瓶颈是 `to_bidiagonal` 和验证阶段的 `Matrix::operator*`。其中 `to_bidiagonal` 在 pthread 版本中占 24.61%, 说明若继续优化总耗时, 应考虑并行化 Householder 上二对角化; `Matrix::operator*` 主要来自正确性验证, 不属于 SVD 核心算法。

仍待补充: 若后续希望更精确地区分 OpenMP 的隐式 barrier、动态调度和线程等待成本, 可以在服务器上额外使用 `perf sched`、`perf lock` 或带调试符号的 libgomp 重新采样; 也可以在代码内加入每线程计时, 从应用层统计 block phase、row-apply phase 和 barrier wait 的占比。

9 结论

本实验围绕 SVD 框架中的 `gkh_svd_from_bidiagonal` 函数进行了 pthread 与 OpenMP 多线程优化。最终实现并不是简单并行 block 循环, 而是采用了 B 矩阵 block-local 更新、Givens 旋转日志、 U/V 延迟批量行更新以及动态任务调度。

在课程服务器三次测试中, 所有版本均通过 5 组正确性测试。对于 1000×1000 大规模样例, pthread 版本将 GKH 阶段平均耗时从 32797.80ms 降至 6088.99ms, OpenMP 版本降至 5475.23ms, 分别取得

约 $5.39\times$ 和 $5.99\times$ 的 GKH 阶段加速。由于上二对角化阶段尚未优化，整体加速比分别为约 $2.84\times$ 和 $3.08\times$ 。

后续若继续提高总性能，优先方向应是并行化 Householder 上二对角化，并补充 profiling 数据来量化同步开销、负载均衡和内存带宽瓶颈。

A 实验文件说明

本次实验相关目录如下：

- `svd_multi/svd_origin`: 原始串行版本。
- `svd_multi/svd_pthread`: pthread 优化版本。
- `svd_multi/svd_openMP`: OpenMP 优化版本。
- 各目录下的 `test/`: 课程服务器自动测试输出。

B 与大模型协作记录

本实验中使用生成式 AI 辅助阅读实验指导书、分析 GKH 迭代并讨论并行化方案。报告提交前应按课程要求整理完整对话记录，作为附录单独提交或附在本文档之后。

参考资料

- [1] 并行程序设计实验指导书：pthread 编程，2026。
- [2] 并行程序设计实验指导书：OpenMP 编程，2026。
- [3] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press.